

CHENNAI INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT-BASED LEARNING (PBL) REPORT
MOBILE APPLICATION DEVELOPMENT (CS4504)

NEURASCAN AI: INTELLIGENT BEHAVIOURAL SCREENING FOR NEUROLOGICAL RISK ANALYSIS

Submitted in partial fulfilment of the requirements for the
Project-Based Learning component of Mobile Application Development

Submitted by

Madhav M S (24CS0491)

M Kiruthick Kannaa (24CS0486)

Under the guidance of

Mr. A. JEBARAJ RATNAKUMAR, M.Tech., (Ph.D)

Assistant Professor, Department of CSE



[Month, Year]

CHENNAI INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT-BASED LEARNING (PBL) REPORT
MOBILE APPLICATION DEVELOPMENT (CS4504)

NEURASCAN AI: INTELLIGENT BEHAVIOURAL SCREENING FOR NEUROLOGICAL RISK ANALYSIS

Submitted in partial fulfilment of the requirements for the
Project-Based Learning component of Mobile Application Development

Submitted by

Madhav M S (24CS0491)

M Kiruthick Kannaa (24CS0486)

Under the guidance of

Mr. A. JEBARAJ RATNAKUMAR, M.Tech., (Ph.D)

Assistant Professor, Department of CSE



[Month, Year]

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

- IM1.** To create next generation leaders by effective teaching learning methodologies and in still scientific park in them to meet the global challenges.
- IM2.** To transform lives through deployment of emerging technology, novelty and sustainability.
- IM3.** To inculcate human values and ethical principles to cater the societal needs.
- IM4.** To contribute towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R&D establishments.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

DM1. To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.

DM2. To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.

DM3. To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.

BONAFIDE CERTIFICATE

This is to certify that the Project-Based Learning report titled **“NeuraScan AI: Intelligent Behavioural Screening for Neurological Risk Analysis”** is a Bonafide record of work carried out by **Madhav M S (24CS0491) and M Kiruthick Kannaa (24CS0486), III Year, CSE – H**, of the **Department of Computer Science and Engineering, Chennai Institute of Technology**, as part of the continuous, mentor-guided Project-Based Learning (PBL) component of the Mobile Application Development course during the academic year **[2026–2027]**. This report reflects the team's work across the review cycles listed in Section 1 below, not a single end-of-term submission.

Faculty Mentor
Mr. A. Jebaraj Ratnakumar, M.Tech., (Ph.D)

Head of the Department
Dr. S. Pavithra, M.E., Ph.D.

Date:

Submitted for the PBL evaluation held on **[date]**.

Faculty Mentor: _____

PBL In-charge: _____

DECLARATION

We declare that this Project-Based Learning report titled **“NeuraScan AI: Intelligent Behavioural Screening for Neurological Risk Analysis”** reflects our own work carried out under the mentorship of **Mr. A. Jebaraj Ratnakumar, M.Tech., (Ph.D), Assistant Professor, Department of CSE**, across the PBL review cycle. All sources of information used have been duly acknowledged and cited.

Team Members:

Madhav M S, 24CS0491, III Year CSE – H

Signature: _____

M Kiruthick Kannaa, 24CS0486, III Year CSE – H

Signature: _____



PBL COURSE AND TEAM DETAILS

Field	Details
Course	Mobile Application Development— Project-Based Learning (PBL)
Faculty Mentor	Mr. A. Jebaraj Ratnakumar, M.Tech., (Ph.D), Assistant Professor, CSE
Section / Team No.	CSE – H (III Year), Team [No.]
PBL Duration	[No. of weeks, e.g. Week 1 – Week 12]
Review Cycles Completed	Review 0 th : date Review 1 st : date [Review 2 nd : date
Project Track / Domain	Healthcare – Digital Neurology (Behavioural Screening)

Team Roles and Responsibilities

Name	Register No.	Role	Primary Responsibility
Madhav M S	24CS0491	Screening Engine and Backend Lead	Screening engine, feature extraction, Firebase and data layer, testing and evaluation; half of the report (Ch. 5, 7, 9, 11–12)
M Kiruthick Kannaa	24CS0486	Mobile App and UI/UX Lead	Flutter screens, navigation and task modules, UI/UX design, literature survey; half of the report (Ch. 1–4, 6, 8, 10)

ACKNOWLEDGEMENT

We would like to thank **Mr. A. Jebaraj Ratnakumar, M.Tech., (Ph.D), Assistant Professor, Department of Computer Science and Engineering**, for mentoring this project across every review cycle of the PBL—from shaping our driving question in the early weeks to pushing us to test the final model properly before submission. The feedback we got after each review changed the direction of our work more than once, and the project is better for it. We're also grateful to **Dr. S. Pavithra, M.E., Ph.D., Head of the Department of Computer Science and Engineering**, for supporting the PBL structure itself, which gave us room to build something iteratively instead of rushing a single final version.



ABSTRACT

Neurological disorders such as Alzheimer's disease, Parkinson's disease and mild cognitive impairment change memory, speech and movement long before a diagnosis, yet specialist screening is episodic, clinic-bound and compared against population norms that hide individual differences. NeuraScan AI is a cross-platform Flutter mobile application that turns a four-minute smartphone session (word recall, reaction time, picture description and spiral tracing, plus in-app typing rhythm) into nine behavioural features. An on-device engine builds a personal baseline with robust median/MAD statistics, fuses four domain deviations with explainable weights, smooths them with an exponentially weighted moving average and raises a notable-deviation status only when the change persists; a three-question context check-in excludes tired or unwell days. The app is offline-first (SQLCipher), syncs only derived scores to Firebase and never diagnoses. In a simulated cohort of 2,400 synthetic users, with alert thresholds calibrated to at most 5% false alerts on healthy users (4.0% observed), the engine detected 98.8% of gradual cognitive declines and 84.4% of motor declines, against 50.0% and 26.4% for conventional population-norm screening. Personal baselines with persistence and context gating make frequent smartphone screening more sensitive without more false alarms.



Keywords: Digital biomarkers, Flutter, Personal baseline, Explainable screening, Neurological risk

TABLE OF CONTENTS

TITLE	PAGE NO.
TEAM AND ROLE RESPONSIBILITIES	vii
ABSTRACT AND KEYWORDS	ix
LIST OF TABLES	xiii
LIST OF FIGURES	xiii
LIST OF ABBREVIATIONS	xiv
CHAPTER 1 – INTRODUCTION	
Section	Page No.
1.1 Project Overview	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope of the Project	2
CHAPTER 2 – LITERATURE SURVEY	
Section	Page No.
2.1 Related Existing Projects	2
2.2 Review of Existing Technologies	3
2.3 Comparison of Existing Systems	3
CHAPTER 3 – EXISTING SYSTEM	
Section	Page No.
3.1 Description	4
3.2 Drawbacks and Limitations	4
CHAPTER 4 – PROPOSED SYSTEM	
Section	Page No.
4.1 Proposed Solution	4
4.2 Features	5
4.3 Advantages	6
CHAPTER 5 – SYSTEM DESIGN	
Section	Page No.
5.1 System Architecture / Block Diagram	6
5.1.1 Physical Design	6
5.1.2 Logical Design	7
5.2 Working Principle	7
5.3 UML / Use Case Diagram	8

Section	Page No.
5.4 Flowchart	9

CHAPTER 6 – DEVELOPMENT TOOLS AND TECHNOLOGIES

Section	Page No.
6.1 Mobile Application Development Platform	10
6.2 Programming Language	10
6.3 User Interface Design	10
6.4 Database	10
6.5 API / Backend Services	11
6.6 Development Device / Emulator	11
6.7 Other Tools and Libraries	11

CHAPTER 7 – SOFTWARE IMPLEMENTATION

Section	Page No.
7.1 Software Requirements	11
7.2 Mobile Application Program	11
7.3 Database / API Implementation	12
7.4 Application Testing / Emulator	13

CHAPTER 8 – MOBILE APPLICATION IMPLEMENTATION

Section	Page No.
8.1 Application Setup and Configuration	13
8.2 Module Integration	13
8.3 Prototype Development	13
8.4 Working of the Mobile Application	14
8.5 Screenshots of the Developed Application	16

CHAPTER 9 – TESTING AND RESULTS

Section	Page No.
9.1 Testing Procedure	16
9.2 Test Cases	17
9.3 Experimental Results	17
9.4 Output Screenshot	19
9.5 Performance Analysis	19

CHAPTER 10 – APPLICATIONS

Section	Page No.
10.1 Application Areas	20
10.2 Real-World Applications	20
10.3 Target User Groups	20

Section	Page No.
10.4 Potential Industry Applications	20

CHAPTER 11 – ADVANTAGES AND LIMITATIONS

Section	Page No.
11.1 Advantages	21
11.2 Limitations	21
11.3 Security Considerations	21
11.4 Performance Constraints	21

CHAPTER 12 – CONCLUSION AND FUTURE SCOPE

Section	Page No.
12.1 Conclusion	22
12.2 Project Outcomes	22
12.3 Future Enhancements	22
12.4 Scope for AI/ML Integration	22
12.5 Scope for Cloud and IoT Integration	22

CHAPTER 13 – REFERENCES

Section	Page No.
References [1]–[20] (IEEE style)	23

CHAPTER 14 – APPENDIX

Section	Page No.
A.1 Full source code	25
A.2 Weekly PBL progress log	25
A.3 Self and Peer Assessment	25
A.4 Project Structure and Source Listings	25
A.5 Screening Engine Reference Implementation	26
A.6 Simulation Parameters	27

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
Figure 5.1	System Architecture / Block Diagram	6
Figure 5.1.1	Physical Design	7
Figure 5.1.2	Logical Design	7
Figure 5.2	Sequence Diagram of a Screening Session	8
Figure 5.3	Use Case Diagram	8
Figure 5.4	Application Flowchart	9
Figure 6.1	Mobile Application Development Platform and Tool-chain	10
Figure 7.1	Entity Relationship Diagram	12
Figure 7.2	Application Testing – Engine Unit Test Execution	13
Figure 8.1	Screenshots of Prototype Development – Wireframe to Final Design	14
Figure 8.2	Login / Registration Screen	15
Figure 8.3	Home Screen / Dashboard	15
Figure 8.4	Application Navigation Screens	15
Figure 8.5	Screenshots of the Developed Mobile Application – Assessment Tasks	16
Figure 9.1	Application Testing Workflow	16
Figure 9.2	Detection Sensitivity of the Five Methods	18
Figure 9.3	Application Performance Analysis – Effect of the Persistence Window	18
Figure 9.4	Application Output Screenshots – Explainable Report and Trends	19
Figure 9.5	Final Application Result – Deviation Trajectory of a Simulated User	20
Figure A.1	Project Structure	25
Figure A.2	Class Diagram	26

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
Table 2.3	Comparison of Existing Mobile Applications / Systems	3
Table 4.1	Functional Requirements of the Proposed Application	5
Table 4.2	Non-Functional Requirements	5
Table 6.1	Software Development Tools and Technologies	11
Table 7.1	Hardware and Software Requirements	11
Table 7.2	Behavioural Features Extracted on the Device	12
Table 7.3	Database Collections and API Operations	12
Table 8.1	Application Screens and Their Functionalities	15

TABLE NO.	TABLE NAME	PAGE NO.
Table 9.1	Test Cases and Expected Results	17
Table 9.2	Test Case Execution Results (Screening Engine Unit Tests)	17
Table 9.3	Experimental Result Comparison	17
Table 9.5	Performance Analysis of the Screening Engine	19
Table A.1	Weekly PBL Progress Log	25
Table A.2	Simulated Population Parameters (mean, between-person SD, within-person SD)	27

LIST OF ABBREVIATIONS

AD – Alzheimer's Disease
API – Application Programming Interface
CV – Coefficient of Variation
DPDP – Digital Personal Data Protection (Act, 2023)
EWMA – Exponentially Weighted Moving Average
FR / NFR – Functional / Non-Functional Requirement
IMU – Inertial Measurement Unit
MAD – Median Absolute Deviation
MCI – Mild Cognitive Impairment
MoCA – Montreal Cognitive Assessment
PBL – Project-Based Learning
PD – Parkinson's Disease
PDF – Portable Document Format
PCM – Pulse-Code Modulation
RMSE – Root Mean Square Error
RT – Reaction Time
SD – Standard Deviation
SDK – Software Development Kit
TFLite – TensorFlow Lite
TLS – Transport Layer Security
UI / UX – User Interface / User Experience
VAD – Voice Activity Detection

CHAPTER 1

INTRODUCTION

1.1 Project Overview

NeuraScan AI is a smartphone application for **early behavioural screening of neurological risk**. It asks the user to complete a short, repeatable session of four tasks — word recall, reaction time, a spoken picture description and spiral tracing — and quietly measures typing rhythm in its own text fields. Each session is reduced on the phone to nine behavioural features across four domains: cognitive, speech, motor and interaction.

The central idea is a **digital behavioural fingerprint**: instead of comparing a person with a population average, NeuraScan first learns what is normal for *that* person over six baseline sessions, and then watches for sustained drift away from it. When drift persists, the app explains which domains contributed and suggests discussing the change with a doctor. The app never names a disease; it is a screening and awareness tool, not a diagnostic device.

1.2 Problem Statement

Neurological disorders are the leading cause of disability-adjusted life-years worldwide and the second leading cause of death [1]. For conditions such as Alzheimer's disease (AD), Parkinson's disease (PD) and mild cognitive impairment (MCI), subtle changes in memory, speech fluency, fine motor control and reaction speed appear years before a formal diagnosis [2]. Current screening misses much of this window for four reasons:

- **Episodic:** a paper test such as the MoCA [4] is taken once, in a clinic, usually after symptoms are noticed.
- **Population-referenced:** a score is judged against age norms, so a high performer can lose a great deal before crossing a cut-off, while a naturally slower person may be flagged falsely.
- **Single-modality:** most tools look at cognition *or* movement *or* speech, not all together.
- **Access:** specialists are concentrated in cities, and repeat visits are costly for older adults.

The problem addressed by this project is therefore: *how can an ordinary smartphone collect frequent, multimodal behavioural measurements and flag a sustained, person-specific change in a way that is sensitive, resistant to false alarms, explainable and privacy-preserving — without claiming to diagnose?*

1.3 Objectives

- Design a Flutter application that delivers a standardised four-minute screening session on Android and iOS.
- Extract nine interpretable behavioural features on the device from touch, microphone and keystroke timing signals.

- Build a personal baseline and a deviation engine that fuses four domains, smooths noise and requires persistence before alerting.
- Reduce false alarms from ordinary bad days through a context check-in and task-level quality gates.
- Present results as an explainable status with per-domain contributions and a shareable PDF report.
- Protect user data with offline-first encrypted storage, opt-in sync of derived scores only, and non-diagnostic language.
- Evaluate the engine quantitatively against population-norm and single-session baselines on a simulated cohort.

1.4 Scope of the Project

In scope: adults (primarily 45 years and above, and anyone with a family history or personal concern) self-screening on their own phone; four active tasks and one passive signal; English and Tamil interfaces; personal baseline tracking; explainable reports; Firebase authentication and opt-in cloud backup of derived scores.

Out of scope: diagnosis of any disease, treatment advice, use with children, clinician dashboards, gait sensing with the inertial measurement unit (IMU), and machine-learning models trained on patient data. These are listed as future work in Chapter 12, because they need clinical data and ethics approval that a PBL project cannot obtain.

CHAPTER 2

LITERATURE SURVEY

2.1 Related Existing Projects

Smartphone digital biomarkers. Kourtis et al. [2] review how phones and wearables can capture behaviour relevant to AD, arguing that continuous, real-world measurement can detect change earlier than clinic visits. Insel [3] frames this as *digital phenotyping*: measuring behaviour from everyday device interaction.

Parkinson's disease. The mPower study [5] collected tapping, voice, gait and memory tasks from thousands of iPhone users through ResearchKit and released the data for research. Arora et al. [6] showed that smartphone voice, posture, gait, finger-tapping and reaction tests could separate PD patients from controls in a pilot study, and Lipsmeier et al. [7] used a smartphone test battery as an exploratory outcome measure in a phase 1 PD drug trial. Giancardo et al. [8] found that key-hold timings from ordinary typing distinguished early PD from controls.

Speech and handwriting. Fraser et al. [9] identified lexical, syntactic and acoustic features of picture descriptions that classified AD speech with high accuracy on the DementiaBank corpus. San Luciano et al. [10] reported that digitised spiral drawing captured motor differences in early

PD. De Jong and Wempe [11] give a transcription-free method for speaking rate from syllable nuclei, which NeuraScan uses so that no speech is converted to text.

Commercial and clinical tools. Altoida [15] offers an augmented-reality cognitive and functional assessment; Linus Health [16] offers digital clock-drawing and recall assessments for primary care. Both are designed for use in clinical settings. Brain-training apps provide engagement but are not screening instruments.

2.2 Review of Existing Technologies

Four families of technique appear repeatedly in the literature and shaped the design:

- **Active tasks** (recall lists, reaction tests, tapping, drawing) give standardised, comparable measurements but are affected by learning, fatigue and device differences.
- **Passive sensing** (typing, touch, IMU) is effortless but noisy and raises greater privacy concerns [3], [8].
- **Robust statistics** such as the median and median absolute deviation (MAD) resist outliers far better than mean and SD when only a few observations exist [12]; control-chart methods such as the EWMA [13] detect small sustained shifts in noisy series.
- **Explainability methods** such as SHAP [14] attribute a model output to its inputs; with an additive fusion model the attribution is exact and needs no approximation.

2.3 Comparison of Existing Systems

Table 2.3 shows the research gap: no accessible tool combines **multimodal smartphone tasks + a personal baseline + explainable, persistence-based alerting + context awareness** in a single self-screening app. NeuraScan AI targets exactly this combination.

Table 2.3: Comparison of Existing Mobile Applications / Systems

System	Modalities	Reference for comparison	Explains result	Setting	Key gap
MoCA (paper) [4]	Cognition	Population cut-off	Item scores	Clinic, one-off	Episodic; needs clinician
mPower [5]	Tapping, voice, gait, memory	Research datasets	Research use only	Research app	Data collection, not screening
Roche PD app [7]	Active tests + passive	Trial arms	For investigators	Clinical trial	Not available to public
Altoida [15]	AR cognitive-functional tasks	Trained model	Clinician report	Clinical	Proprietary, clinic use
Linus Health DCR [16]	Clock drawing, recall	Normative model	Clinician report	Primary care	Cognition only; clinic
NeuraScan AI	Memory, RT, speech, spiral, typing	Own baseline	Per-domain share	Self-screening	Needs clinical validation

CHAPTER 3

EXISTING SYSTEM

3.1 Description

Today, neurological screening in India typically follows a reactive path. A family member notices forgetfulness or a tremor; the person visits a general physician, who may administer a brief paper test such as the MMSE or MoCA [4] and refer to a neurologist. The specialist repeats cognitive and motor examinations, may order imaging, and schedules follow-up months later. Each assessment is a single snapshot scored against age- and education-based norms.

Digital alternatives exist (Chapter 2) but are either research data-collection apps, clinical-trial instruments or clinician-administered tablet tests. Consumer "brain" apps measure performance but do not track deviation from a person's own normal or explain what changed.

Existing workflow

Symptom noticed → GP visit → paper test → referral → specialist examination → imaging / labs → follow-up after 3–6 months.

3.2 Drawbacks and Limitations

- **Late entry:** screening starts only after symptoms are obvious to others.
- **Sparse data:** one or two measurements a year cannot separate a real trend from a bad day.
- **Norm-based thresholds:** individual variation is often larger than early change, which causes both misses and false alarms (quantified in Chapter 9).
- **Single domain:** cognitive tests ignore motor and speech signs and vice versa.
- **Opaque digital scores:** apps that output one number give the user nothing to act on or discuss.
- **Privacy risk:** apps that upload raw audio or keystrokes create sensitive data stores.
- **Cost and access:** travel, specialist fees and waiting time weigh on older and rural users.

CHAPTER 4

PROPOSED SYSTEM

4.1 Proposed Solution

NeuraScan AI moves screening from the clinic to the phone and from population norms to the individual. Its design rests on five decisions, each addressing a drawback from Chapter 3:

- **Frequent, short sessions:** a four-minute session every two to three days gives a dense time series instead of yearly snapshots.
- **Personal baseline:** two familiarisation sessions are discarded (learning effect), then six valid sessions are summarised by the median and MAD of each feature and frozen, so slow decline is not absorbed into "normal".
- **Explainable fusion:** each feature becomes a robust z-score; features average into four domain scores; only worsening counts, weighted 35 / 25 / 25 / 15 % (cognitive, speech, motor, interaction). The share of each domain in the total is the explanation.
- **Persistence, not spikes:** an EWMA ($\lambda = 0.3$) smooths the index, and a *notable deviation* is raised only when it stays above threshold for three consecutive valid sessions.
- **Context gating (new idea):** a three-question check-in (sleep, fatigue, illness or medication change) marks confounded days, which are stored but excluded from the baseline and the trend.

We deliberately chose an interpretable statistical engine over a deep-learning classifier: there is no labelled longitudinal dataset from our target users, and a classifier trained on public case-control data would learn population differences, which is the very problem we set out to avoid. The architecture keeps a TensorFlow Lite slot for learned feature extractors once real data exist (Chapter 12).

4.2 Features

Table 4.1: Functional Requirements of the Proposed Application

ID	Requirement
FR1	Register and sign in with email/password or Google (Firebase Authentication).
FR2	Capture informed consent, dominant hand, language and privacy choices (sync on/off).
FR3	Run a context check-in before every session.
FR4	Deliver memory (8 words, rotating lists), reaction (10 trials), speech (20 s) and spiral (3 turns) tasks.
FR5	Record inter-key timing inside NeuraScan text fields only.
FR6	Apply quality gates and mark invalid sessions.
FR7	Build and freeze a personal baseline; show progress while building.
FR8	Compute domain scores, deviation index, EWMA and status after each session.
FR9	Show dashboard, per-domain trends and an explainable report with advice text.
FR10	Export a PDF report and share it through the OS share sheet.
FR11	Schedule local reminders (default every 2 days).
FR12	Export all data as JSON and delete the account and all data.

Table 4.2: Non-Functional Requirements

Attribute	Requirement
Privacy	Raw audio and touch traces never leave the phone; audio is deleted after feature extraction. Only derived features sync, and only if the user opts in.
Security	SQLCipher AES-256 local database with key in Android Keystore / iOS Keychain; TLS in transit; Firestore rules restrict every document to its owner.
Performance	Complete session ≤ 5 min; engine update < 50 ms on a mid-range phone; cold start < 3 s (targets).
Usability	Touch targets ≥ 48 dp, text scaling to 200 %, spoken instructions, English and Tamil.
Reliability	Offline-first; sync queue retries with back-off; no data loss if the app is killed mid-session.
Portability	Android 8.0 (API 26)+ and iOS 13+ from one Flutter code base.
Ethics	No diagnostic words; every result screen carries a "not a diagnosis" note and a consult-a-

Attribute	Requirement
	doctor path.

4.3 Advantages

- Detects change relative to the person, so both high and low performers are judged fairly.
- Four modalities in one short session on a device the user already owns.
- Explanations are exact (additive model), not post-hoc approximations.
- Persistence and context gating cut false alarms from bad days.
- Works offline; privacy is enforced by architecture, not only by policy.
- Cross-platform Flutter code base keeps development and maintenance cost low.

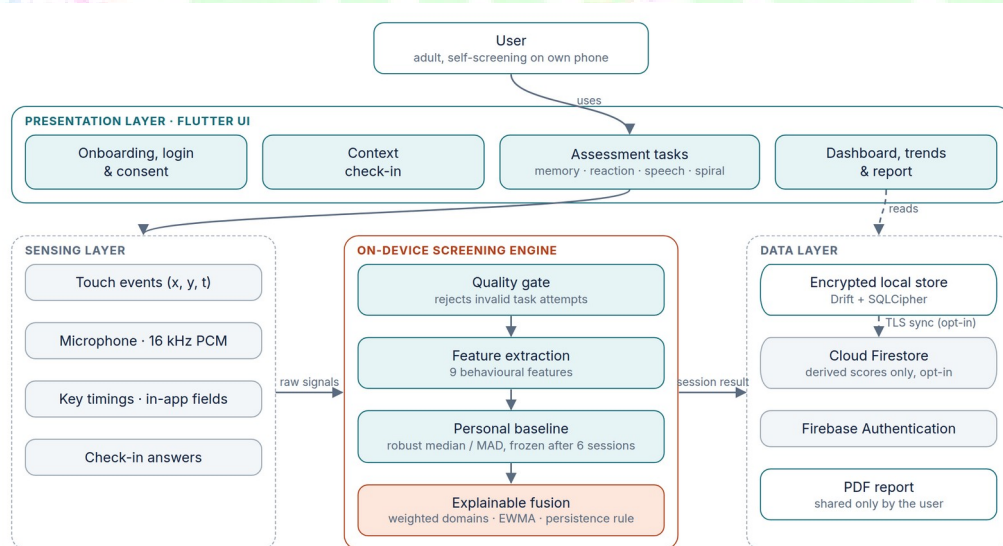
CHAPTER 5

SYSTEM DESIGN

5.1 System Architecture / Block Diagram

Figure 5.1 shows the four layers. The **presentation layer** (Flutter widgets) guides the user; the **sensing layer** captures touch coordinates with timestamps, 16 kHz microphone audio, key timings and check-in answers; the **on-device screening engine** validates, extracts features, compares with the baseline and fuses the result; the **data layer** stores results in an encrypted local database and, if permitted, syncs derived scores to Cloud Firestore.

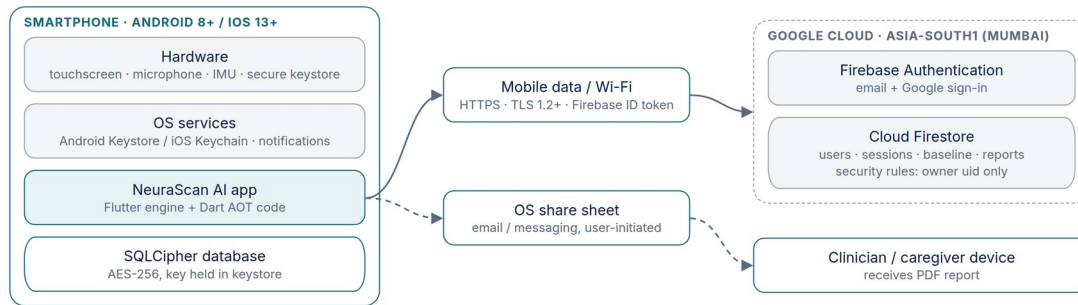
Figure 5.1: System Architecture / Block Diagram



5.1.1 Physical Design

Physically (Figure 5.1.1), everything runs on the user's smartphone. The only network traffic is authentication and the opt-in upload of derived scores to Firebase in the Mumbai region, over TLS. A report reaches a clinician only when the user exports the PDF through the phone's share sheet.

Figure 5.1.1: Physical Design

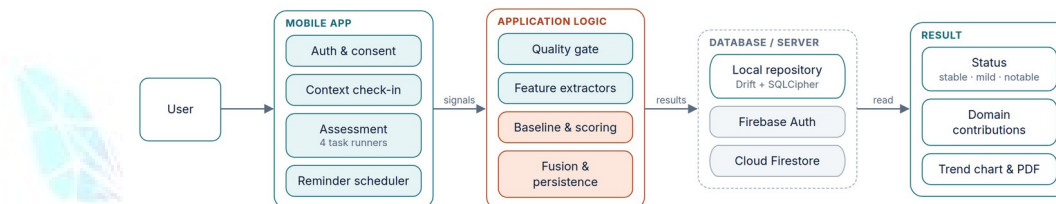


5.1.2 Logical Design

User → Mobile App → Application Logic → Database/Server → Result

Logically (Figure 5.1.2), the app follows the flow above: UI modules collect data, the application logic turns it into a result, the repository persists it, and the result modules present it.

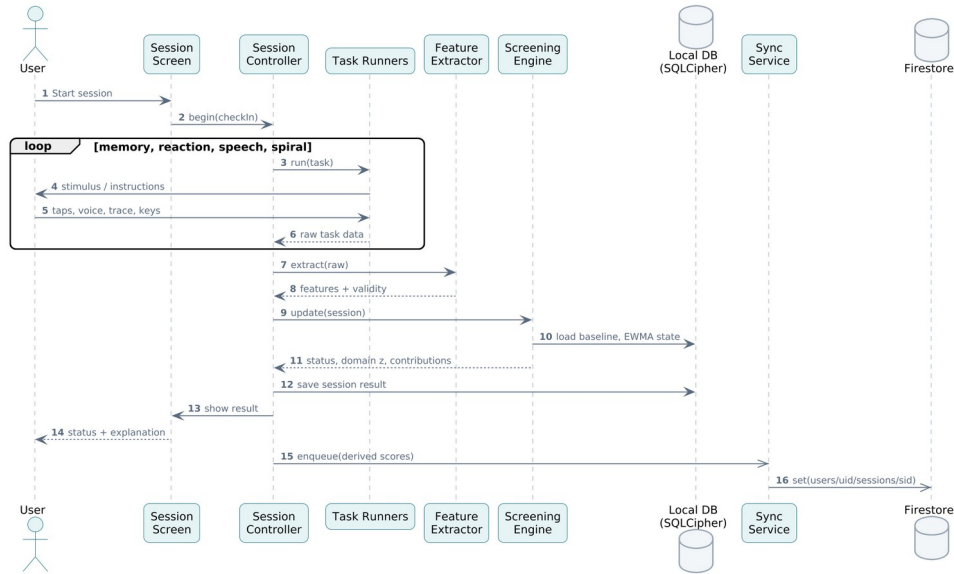
Figure 5.1.2: Logical Design



5.2 Working Principle

On first launch the user sees onboarding, signs in and gives consent. Each session starts with the context check-in, then memory encoding, reaction time, speech and spiral tasks, and finally delayed recall of the words. Each task passes a quality gate; the nine features are extracted; if the baseline is still being built the session is added to the pool, otherwise the engine computes the domain z-scores, deviation index, EWMA and status. The result is saved locally, queued for sync and shown with its explanation. Figure 5.2 gives the message sequence.

Figure 5.2: Sequence Diagram of a Screening Session



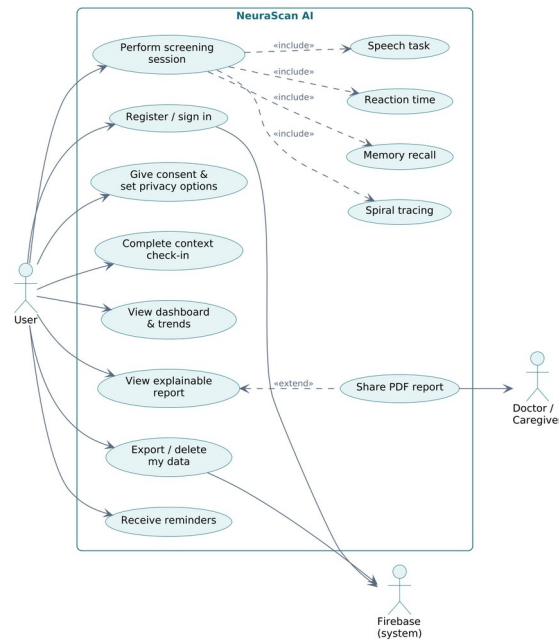
Basic Working Flow:

Start → Login / Register → Consent → Dashboard → Check-in → Four tasks → Delayed recall → Quality gate → Features → Baseline / Engine → Local DB → Sync → Result and trend → End

5.3 UML / Use Case Diagram

The primary actor is the user. The doctor or caregiver is a secondary actor who only receives a shared report, and Firestore is a system actor for authentication and data deletion. "Perform screening session" includes the four tasks; "Share PDF report" extends "View explainable report".

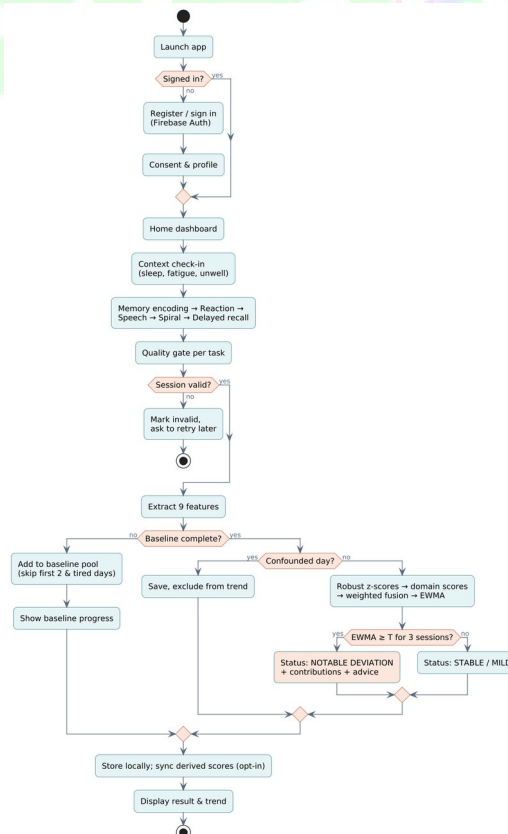
Figure 5.3: Use Case Diagram



5.4 Flowchart

Figure 5.4 shows the decision logic: sign-in check, session validity, baseline completeness, context confound and the persistence test that decides between *stable/mild* and *notable deviation*.

Figure 5.4: Application Flowchart



Simple Flow:

Start → Signed in? → Home → Check-in → Tasks → Valid? → Features → Baseline ready?
→ Confounded? → Score, fuse, smooth → Persistent? → Status → Store and display → End

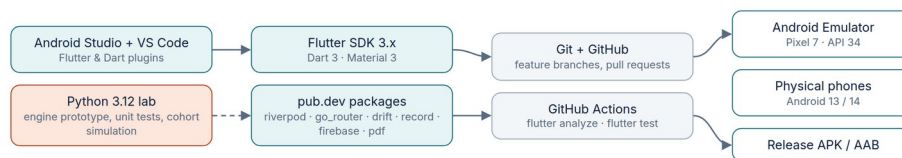
CHAPTER 6

DEVELOPMENT TOOLS AND TECHNOLOGIES

6.1 Mobile Application Development Platform

The app is developed in **Android Studio** (with the Flutter and Dart plug-ins) and **VS Code**, using the Flutter SDK 3.x. Figure 6.1 shows the tool-chain, from the IDE through the Python lab used to prototype the engine, to CI on GitHub Actions and the emulator and devices used for testing.

Figure 6.1: Mobile Application Development Platform and Tool-chain



6.2 Programming Language

Dart 3 is used for the whole app: it compiles ahead-of-time to native ARM code, has sound null safety, and its `async/await` model suits timed tasks. Records and enhanced enums keep feature definitions compact. **Python 3.12** (NumPy, pytest) is used only off-device, to prototype, test and evaluate the screening engine before porting it to Dart; **Kotlin** and **Swift** appear only in the generated platform runners.

6.3 User Interface Design

The UI is written in **Flutter widgets with Material 3**, seeded from the brand colour #146C7A. Screens use Scaffold, NavigationBar and cards; the spiral task uses a CustomPainter with a GestureDetector. Navigation is declarative with `go_router`, state is held in **Riverpod** providers, and text is localised with intl (English, Tamil).

6.4 Database

The primary store is **SQLite through Drift** (type-safe queries and migrations), encrypted with **SQLCipher**; the key is generated once and kept in secure storage (Android Keystore / iOS Keychain). **Cloud Firestore** holds an optional backup of derived scores (Section 7.3).

6.5 API / Backend Services

Firebase Authentication (email/password and Google) issues ID tokens; **Cloud Firestore** is accessed through the FlutterFire SDK, which talks to Google's APIs over HTTPS. There is no custom server: access control is enforced by Firestore Security Rules, which keeps the attack surface small.

6.6 Development Device / Emulator

Testing targets the **Android Emulator (Pixel 7, API 34)** for layout and flow, and **physical Android 13/14 phones** for timing-sensitive tasks, since emulators add touch and audio latency that would distort reaction-time and speech features.

6.7 Other Tools and Libraries

Table 6.1: Software Development Tools and Technologies

Category	Tool / package	Purpose
Framework	Flutter 3.x, Dart 3, Material 3	Cross-platform UI and logic
State / navigation	flutter_riverpod, go_router	Dependency injection, declarative routes
Storage	drift, sqlcipher_flutter_libs, flutter_secure_storage	Encrypted local database and key storage
Cloud	firebase_auth, cloud_firestore	Sign-in and opt-in backup
Sensing	record (16 kHz PCM), GestureDetector	Audio capture, touch traces
Signal processing	Dart port of VAD, syllable-nuclei and FFT routines	Speech and tremor features
Output	fl_chart, pdf, share_plus	Trends, PDF report, sharing
Reminders / i18n	flutter_local_notifications, intl	Session reminders, English and Tamil
Quality	flutter_test, integration_test, pytest, GitHub Actions	Unit, widget, integration tests; CI
Version control	Git, GitHub	Branches, pull requests, reviews

CHAPTER 7

SOFTWARE IMPLEMENTATION

7.1 Software Requirements

Table 7.1: Hardware and Software Requirements

Item	Development	Target device
Processor / RAM	Intel i5 / Ryzen 5, 16 GB RAM	64-bit ARM, 3 GB RAM or more
Operating system	Windows 11, macOS 14 or Ubuntu 22.04	Android 8.0+ (API 26), iOS 13+
SDKs	Flutter 3.x, Dart 3, Android SDK 34, JDK 17, Xcode 15 (iOS)	—
Tools	Android Studio, VS Code, Git, Python 3.12	—
Sensors	—	Touchscreen, microphone
Cloud	Firebase project (Spark plan), region asia-south1	Internet only for sign-in and sync

7.2 Mobile Application Program

The code is organised feature-first (project structure and full listings in Appendix A.4). Each task implements a common TaskRunner interface, a FeatureExtractor turns raw task data into the nine features of Table 7.2, and the ScreeningEngine holds the baseline and EWMA

state (class diagram, Figure A.2). The reaction-trial timing code and the Dart screening engine are listed in Appendix A.4.

Table 7.2: Behavioural Features Extracted on the Device

Domain (weight)	Feature	Computation	Worse if
Cognitive (35 %)	Delayed recall	Words recalled after ~3 min ÷ 8 (edit distance ≤ 1 accepted)	Lower
	Reaction median	Median of 10 trials, random 1–4 s wait	Higher
	Reaction CV	SD ÷ mean of valid trials	Higher
Speech (25 %)	Speaking rate	Syllable nuclei per minute [11] from 20 s description	Lower
	Pause ratio	Silent time in pauses ≥ 250 ms ÷ total (energy VAD)	Higher
Motor (25 %)	Spiral RMSE	RMS radial distance of trace from guide spiral (dp)	Higher
	Tremor index	Share of radial-error power in 4–12 Hz band	Higher
Interaction (15 %)	Inter-key interval	Median gap between character inserts in app fields	Higher
	Inter-key CV	SD ÷ mean of inter-key gaps	Higher

7.3 Database / API Implementation

Locally, Drift defines five tables that mirror the entities in Figure 7.1. In the cloud, the same entities are Firestore documents nested under the user, so a single security rule covers all of them. Table 7.3 lists the operations the app performs; the rules are:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    match /users/{uid}/{doc=**} {
      allow read, write: if request.auth != null && request.auth.uid == uid;
    }
  }
}
```

Figure 7.1: Entity Relationship Diagram

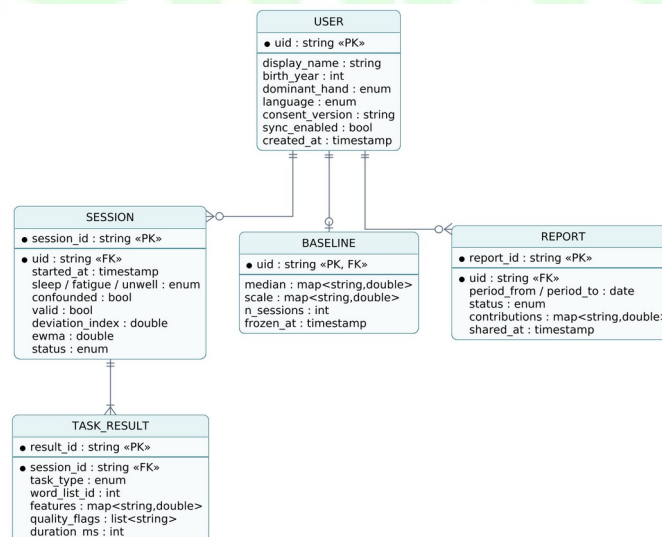


Table 7.3: Database Collections and API Operations

Path (Firestore)	Operation	Content
users/{uid}	set on sign-up; update on settings	Profile, consent version, sync flag
users/{uid}/sessions/{sid}	set after each session (queued)	Context, validity, 9 features, domain z, index, EWMA, status (≈ 0.5 KB)
users/{uid}/baseline/current	set once when frozen	Median and scale per feature

Path (Firestore)	Operation	Content
users/{uid}/reports/{rid}	set on PDF export	Period, status, contributions (PDF stays on phone)
Auth: signIn / signIn / delete	Firebase Auth SDK	ID token; account deletion triggers data wipe

7.4 Application Testing / Emulator

Before any UI work, the engine logic was tested in isolation. Figure 7.2 shows the actual run of the 12 unit tests (listed in Table 9.2) against the reference engine; the same cases are mirrored as Dart unit tests executed by `flutter test` on every push. Layout and flows are checked on the emulator, and timing tasks on physical phones.

Figure 7.2: Application Testing – Engine Unit Test Execution

```
$ python -m pytest -v test_engine.py
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
rootdir: ~/neurascan/engine_lab
collecting ... collected 12 items

test_engine.py::test_ut01_quality_gate_rejects_anticipations PASSED [ 8%]
test_engine.py::test_ut02_quality_gate_rejects_short_speech_and_partial_spiral PASSED [ 16%]
test_engine.py::test_ut03_familiarisation_sessions_not_in_baseline PASSED [ 25%]
test_engine.py::test_ut04_baseline_uses_robust_median PASSED [ 33%]
test_engine.py::test_ut05_scale_floor_prevents_division_by_zero PASSED [ 41%]
test_engine.py::test_ut06_orientation_lower_recall_is_worse PASSED [ 50%]
test_engine.py::test_ut07_improvement_does_not_raise_index PASSED [ 58%]
test_engine.py::test_ut08_contributions_sum_to_one PASSED [ 66%]
test_engine.py::test_ut09_confounded_session_excluded PASSED [ 75%]
test_engine.py::test_ut10_single_bad_session_does_not_alert PASSED [ 83%]
test_engine.py::test_ut11_sustained_deviation_alerts PASSED [ 91%]
test_engine.py::test_ut12_invalid_session_ignored PASSED [100%]

===== 12 passed in 0.02s =====
```

CHAPTER 8

MOBILE APPLICATION IMPLEMENTATION

8.1 Application Setup and Configuration

The project was created with `flutter create --org in.citchennai neurascan`, minimum SDK set to API 26, and microphone permission declared in `AndroidManifest.xml` and `Info.plist` with a plain-language purpose string. Firebase was added with `flutterfire configure`, generating `firebase_options.dart`. Packages from Table 6.1 were added to `pubspec.yaml`; `flutter analyze` with the recommended lints runs in CI.

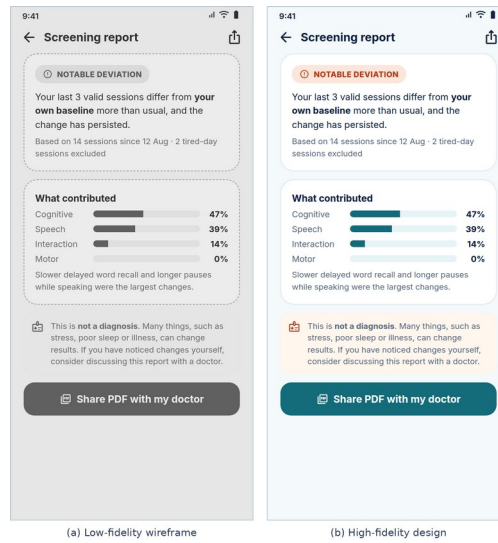
8.2 Module Integration

Modules are wired through Riverpod providers: the `SessionController` depends on the task runners, feature extractor, engine and repository; the UI only watches controller state. The repository writes to Drift first and then enqueues a sync job, so the UI never waits for the network. Auth state drives `go_router` redirects (signed-out users go to login; users without consent go to the consent screen).

8.3 Prototype Development

The UI was prototyped in two steps: grey-scale low-fidelity wireframes to fix layout and flow, then high-fidelity screens in the app theme (Figure 8.1).

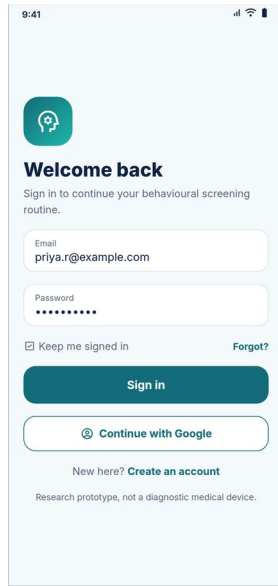
Figure 8.1: Screenshots of Prototype Development – Wireframe to Final Design



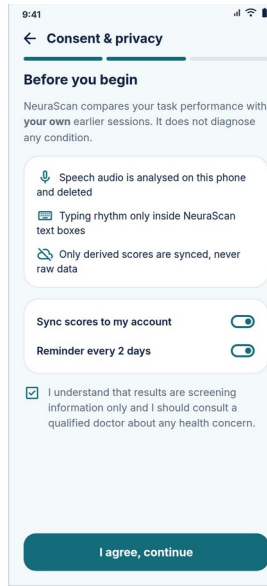
8.4 Working of the Mobile Application

The user signs in and gives consent (Figure 8.2), lands on the home dashboard (Figure 8.3), answers the check-in, completes the four tasks and delayed recall (Figure 8.5) and sees the explainable report (Figure 9.4). Figure 8.4 shows the complete navigation and Table 8.1 the purpose of each screen.

Figure 8.2: Login / Registration Screen

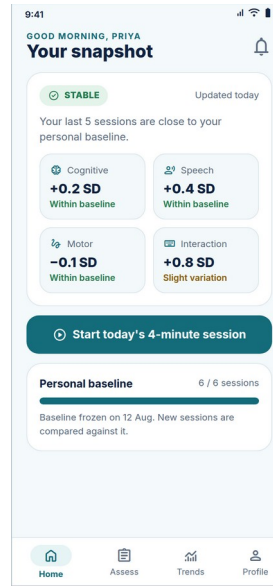


(a) Login

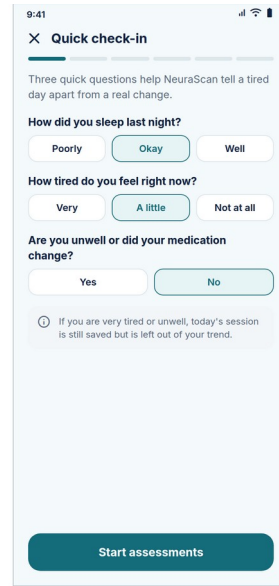


(b) Consent & privacy

Figure 8.3: Home Screen / Dashboard



(a) Home dashboard



(b) Context check-in

Figure 8.4: Application Navigation Screens

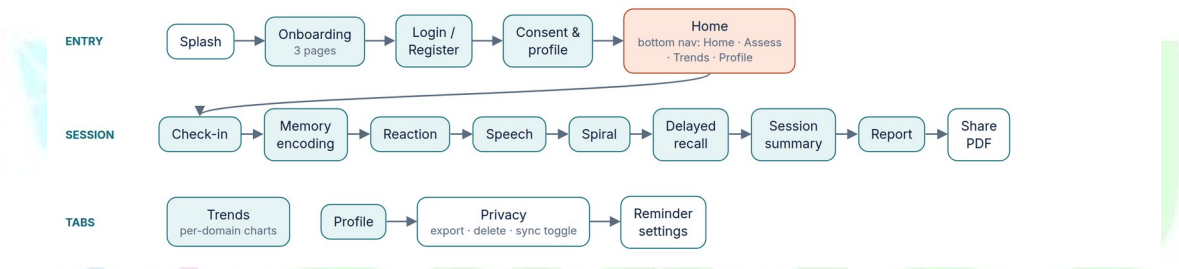
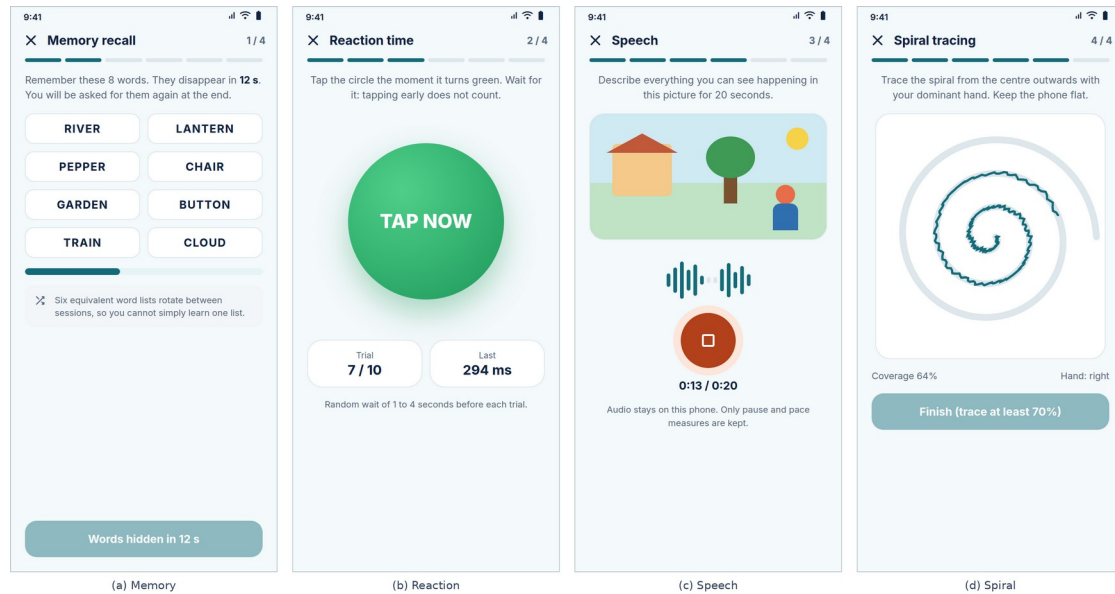


Table 8.1: Application Screens and Their Functionalities

Screen	Functionality
Splash, onboarding	Brand, disclaimer, three-page explanation of the personal-baseline idea
Login / register, consent	Firestore sign-in; consent, hand, language, sync and reminder settings
Home	Status, domain cards, baseline progress, start session
Check-in	Sleep, fatigue, illness / medication change
Memory, reaction, speech, spiral, delayed recall	Timed tasks with instructions read aloud and quality gates
Summary, report	Status, contributions, explanation, PDF share
Trends, profile, privacy	Per-domain charts; reminders; export and delete data

8.5 Screenshots of the Developed Application

Figure 8.5: Screenshots of the Developed Mobile Application – Assessment Tasks



Screens rendered from the NeuraScan AI UI design at 360×760 dp.

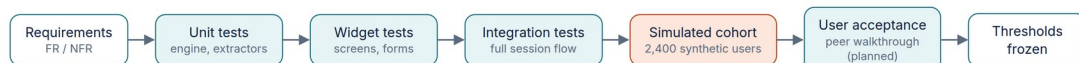
CHAPTER 9

TESTING AND RESULTS

9.1 Testing Procedure

Testing followed the pipeline in Figure 9.1. Unit tests cover the engine and extractors; widget tests cover forms and screen states; integration tests drive a full session on the emulator. Because no longitudinal patient data exist, the engine was evaluated on a **simulated cohort**: each synthetic user has personal trait levels drawn from a population, day-to-day noise smaller than the between-person spread, a practice effect in early sessions, and tired days (15 % of sessions, 80 % of them reported in the check-in). Three groups were simulated over 60 sessions: healthy, gradual cognitive decline and gradual motor decline starting at session 25 and reaching 3 within-person SD by session 60. Thresholds were calibrated on one cohort (300 users per group) to allow at most 5 % of healthy users to be alerted, then reported on a fresh cohort (500 per group).

Figure 9.1: Application Testing Workflow



9.2 Test Cases

Table 9.1: Test Cases and Expected Results

ID	Module	Input / action	Expected result
TC1	Auth	Wrong password	Inline error, no navigation
TC2	Consent	Continue without ticking agreement	Button disabled
TC3	Check-in	Fatigue = "Very"	Session saved as confounded, excluded from trend
TC4	Reaction	Tap before stimulus 3 times	Session marked invalid, retry offered
TC5	Speech	Silence for 20 s	Voiced < 8 s → invalid
TC6	Spiral	Trace 50 % of guide	Finish disabled until coverage ≥ 70 %
TC7	Baseline	Complete sessions 1–8	Progress 0 → 6/6; baseline frozen after session 8
TC8	Engine	One very poor session	Status not "notable"
TC9	Sync	Complete session offline, reconnect	Document appears in Firestore once
TC10	Privacy	Delete account	Local DB wiped, Firestore documents removed, signed out

Table 9.2: Test Case Execution Results (Screening Engine Unit Tests)

ID	Test	Result
UT1–2	Quality gate rejects > 2 anticipations, < 8 s voiced speech, < 70 % spiral coverage	Pass
UT3	Familiarisation sessions excluded from baseline	Pass
UT4–5	Median resists an outlier; scale floor prevents division by zero	Pass
UT6–7	Lower recall raises cognitive z; improvements never raise the index	Pass
UT8	Domain contributions sum to 1	Pass
UT9	Confounded session leaves EWMA unchanged	Pass
UT10–11	One bad session does not alert; sustained deviation does	Pass
UT12	Invalid session ignored	Pass

All 12 tests passed (Figure 7.2). A consistency check also confirmed that the stateful engine and the batch evaluation code raised alerts at identical sessions for 100 of 100 simulated users.

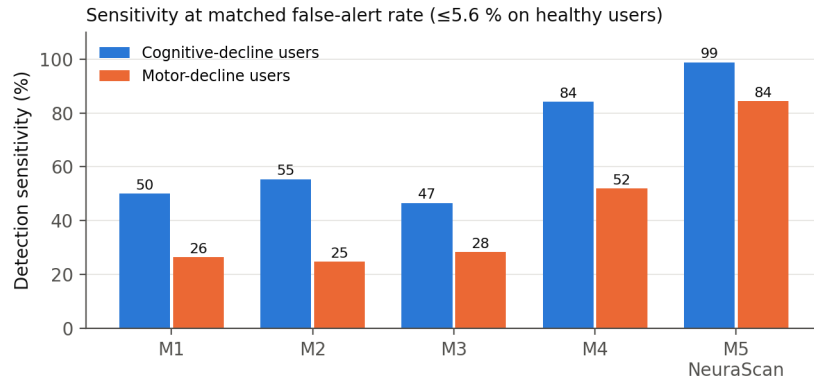
9.3 Experimental Results

Five methods were compared at matched false-alert levels (Table 9.3, Figure 9.2). *Sensitivity* is the share of declining users alerted after onset; *delay* is the median number of sessions from onset to alert.

Table 9.3: Experimental Result Comparison

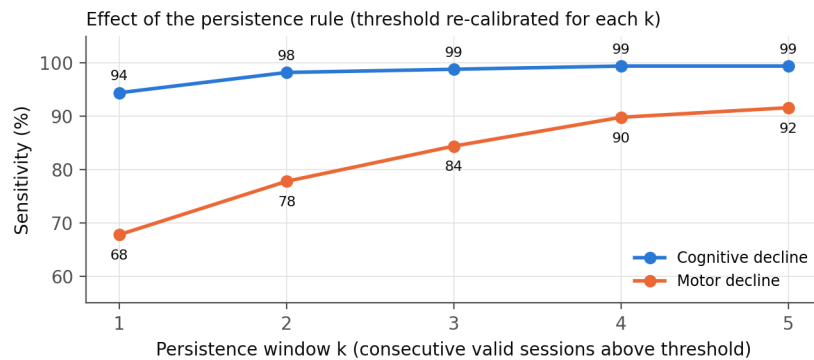
Method	False alerts (healthy)	Sensitivity – cognitive	Sensitivity – motor	Median delay (sessions) cog / motor
M1 Population norms, single session	5.6%	50.0%	26.4%	24 / 24
M2 Population norms + EWMA + persistence	5.6%	55.4%	24.8%	25 / 27
M3 Personal baseline, single session	3.4%	46.6%	28.4%	23 / 25
M4 Personal baseline + EWMA + persistence	4.4%	84.2%	52.0%	23 / 25
M5 NeuraScan (M4 + context check-in)	4.0%	98.8%	84.4%	18 / 23

Figure 9.2: Detection Sensitivity of the Five Methods



Population norms (M1, M2) miss most declines because a user who starts well above average can decline a long way and still look "normal". A personal baseline alone (M3) does not help if every session is judged in isolation: noise forces a high threshold. Combining the baseline with smoothing and persistence (M4) lifts sensitivity to 84.2% and 52.0%, and the context check-in (M5) removes tired-day noise, reaching **98.8%** and **84.4%** with 4.0% false alerts. NeuraScan also alerts earlier (median 18 sessions after onset for cognitive decline), when the simulated change is only about 1.6 of the user's own day-to-day SD — well inside the population normal range. The top contributing domain matched the simulated domain in 66.4% (cognitive) and 68.2% (motor) of alerts; cognitive decline also slows speech and typing, which is why the report shows all four contributions rather than a single label.

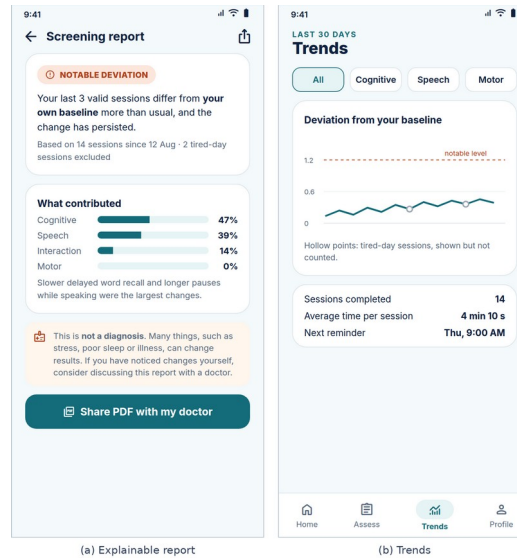
Figure 9.3: Application Performance Analysis – Effect of the Persistence Window



Requiring persistence helps most for motor decline, whose features are noisier (Figure 9.3). Sensitivity keeps rising slightly up to $k = 5$, but a larger k also means a longer minimum wait before any alert (five sessions is about ten days). We keep $k = 3$ as a conservative default and expose k as a configuration value to be re-tuned once real longitudinal data are available.

9.4 Output Screenshot

Figure 9.4: Application Output Screenshots – Explainable Report and Trends



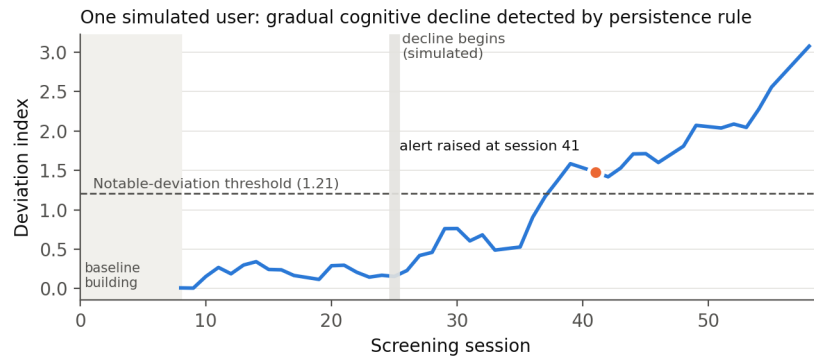
9.5 Performance Analysis

Table 9.5: Performance Analysis of the Screening Engine

Measure	Value	Comment
Engine update time	5.4 μ s	Python reference, one session; far below the 50 ms target
Baseline state	20 numbers	Median and scale per feature, plus EWMA and run count
Synced record size	$\approx$ 0.5 KB per session	JSON of derived fields; no raw signals
Designed session length	$\approx$ 4 min	Check-in 15 s, memory 45 s, reaction 40 s, speech 30 s, spiral 40 s, recall 30 s
Persistence k = 1	94.4% / 67.8%	Sensitivity cog / motor at 4.2% false alerts
Persistence k = 3 (default)	98.8% / 84.4%	at 4.0% false alerts
Persistence k = 5	99.4% / 91.6%	at 4.6% false alerts

Figure 9.5 follows one simulated user from the cognitive-decline group through the whole period. The index stays near zero during the baseline, drifts upward after onset and crosses the threshold; the alert is raised at session 41 once the persistence rule is met, with contributions of 47 % cognitive, 39 % speech and 14 % interaction — the values shown on the report screen in Figure 9.4.

Figure 9.5: Final Application Result – Deviation Trajectory of a Simulated User



CHAPTER 10

APPLICATIONS

10.1 Application Areas

- **Personal self-monitoring** for adults over 45, or anyone with a family history of AD or PD, who want an early, private signal.
- **Caregiver support:** a family member can see a shared PDF instead of relying on impressions.
- **Pre-consultation data:** a doctor receives months of trend data rather than a single clinic score.

10.2 Real-World Applications

- **Community and primary-care programmes:** health workers can run sessions on a shared phone during home visits (one profile per person).
- **Remote follow-up** after a diagnosis, to show whether a person is stable between appointments.
- **Research and clinical trials:** frequent, low-cost digital measures of the kind used in [5] and [7].

10.3 Target User Groups

Older adults and their families, general physicians and neurologists (as report recipients), community health programmes, and research teams. Children are excluded.

10.4 Potential Industry Applications

Digital-health and telemedicine platforms can embed the engine as a screening module; pharmaceutical sponsors can use it as an exploratory remote endpoint; hospital networks can offer it for post-discharge monitoring. Any such use would require clinical validation and regulatory review first.

CHAPTER 11

ADVANTAGES AND LIMITATIONS

11.1 Advantages

- Personal baselines found far more simulated declines than population norms at the same false-alert rate (Table 9.3).
- Four modalities in about four minutes on an ordinary phone, with no extra hardware.
- Exact, additive explanations that users and doctors can read.
- Robust to bad days through the check-in, quality gates and persistence rule.
- Privacy by design: on-device processing, encrypted storage, opt-in sync of derived scores only.

11.2 Limitations

- **No clinical validation.** All quantitative results come from a simulated cohort whose parameters we chose; they show the engine behaves as designed, not that it detects disease in real people.
- **Learning and device effects.** Rotating word lists and familiarisation sessions reduce, but do not remove, practice effects; a new phone changes touch and audio characteristics and needs a re-baseline.
- **Language and literacy.** Speech and typing features depend on language and education; only English and Tamil are planned.
- **Fixed weights.** Domain weights (35/25/25/15 %) are chosen from the literature, not learned from data.
- **Self-report.** Context gating works only if users answer the check-in honestly.

11.3 Security Considerations

Behavioural data about possible neurological decline is sensitive health-related data. Threats considered: a lost phone (mitigated by SQLCipher and OS lock), token theft (short-lived Firebase ID tokens, TLS), cross-user access (owner-only Firestore rules), and over-collection (no raw audio, no system-wide keyboard logging). The design follows the consent, purpose-limitation and erasure principles of India's Digital Personal Data Protection Act, 2023 [20].

11.4 Performance Constraints

Timing features depend on display refresh and touch-sampling rates (60–120 Hz) and on audio latency, which vary between phones; this is why emulators are not used for timing tests and why baselines are per device. On-device signal processing is light (short FFTs and energy frames), so battery and CPU cost per session are small.

CHAPTER 12

CONCLUSION AND FUTURE SCOPE

12.1 Conclusion

NeuraScan AI shows how a cross-platform mobile application can make neurological risk screening frequent, personal, explainable and private. The main lesson from our evaluation is that *how* a phone judges change matters as much as *what* it measures: the same features detected 50.0% of simulated cognitive declines against population norms and 98.8% against a personal baseline with persistence and context gating, at matched false-alert rates.

12.2 Project Outcomes

- A complete mobile application design: architecture, data model, navigation and ten screens.
- A tested screening engine (12/12 unit tests) with a Dart implementation plan and exact explanations.
- A reproducible simulated-cohort evaluation comparing five methods.
- A privacy and ethics model that keeps the product non-diagnostic.

12.3 Future Enhancements

- Pilot study with volunteers (with institutional ethics approval) to measure test-retest reliability and usability.
- Gait and postural tremor tasks using the accelerometer and gyroscope.
- Slow, guarded re-baselining when a user stays stable for a long period; per-device baselines.
- Clinician web view that receives reports only with user consent.

12.4 Scope for AI/ML Integration

Once real longitudinal data exist, learned models can replace hand-built parts: an on-device TensorFlow Lite model for richer speech features, anomaly detectors (isolation forest or autoencoders) on the feature vector, and learned domain weights, with SHAP [14] keeping explanations. Federated learning would let models improve without collecting raw data.

12.5 Scope for Cloud and IoT Integration

Smartwatches can add passive gait, sleep and tremor data; Cloud Functions can produce aggregate, anonymised statistics for research with consent; and FHIR-based export can place reports into hospital record systems.

CHAPTER 13

REFERENCES

- [1] GBD 2016 Neurology Collaborators, "Global, regional, and national burden of neurological disorders, 1990–2016: a systematic analysis for the Global Burden of Disease Study 2016," *Lancet Neurol.*, vol. 18, no. 5, pp. 459–480, 2019.
- [2] L. C. Kourtis, O. B. Regele, J. M. Wright, and G. B. Jones, "Digital biomarkers for Alzheimer's disease: the mobile/wearable devices opportunity," *npj Digit. Med.*, vol. 2, art. 9, 2019.
- [3] T. R. Insel, "Digital phenotyping: technology for a new science of behavior," *JAMA*, vol. 318, no. 13, pp. 1215–1216, 2017.
- [4] Z. S. Nasreddine *et al.*, "The Montreal Cognitive Assessment, MoCA: a brief screening tool for mild cognitive impairment," *J. Am. Geriatr. Soc.*, vol. 53, no. 4, pp. 695–699, 2005.
- [5] B. M. Bot *et al.*, "The mPower study, Parkinson disease mobile data collected using ResearchKit," *Sci. Data*, vol. 3, art. 160011, 2016.
- [6] S. Arora *et al.*, "Detecting and monitoring the symptoms of Parkinson's disease using smartphones: a pilot study," *Parkinsonism Relat. Disord.*, vol. 21, no. 6, pp. 650–653, 2015.
- [7] F. Lipsmeier *et al.*, "Evaluation of smartphone-based testing to generate exploratory outcome measures in a phase 1 Parkinson's disease clinical trial," *Mov. Disord.*, vol. 33, no. 8, pp. 1287–1297, 2018.
- [8] L. Giancardo *et al.*, "Computer keyboard interaction as an indicator of early Parkinson's disease," *Sci. Rep.*, vol. 6, art. 34468, 2016.
- [9] K. C. Fraser, J. A. Meltzer, and F. Rudzicz, "Linguistic features identify Alzheimer's disease in narrative speech," *J. Alzheimer's Dis.*, vol. 49, no. 2, pp. 407–422, 2016.
- [10] M. San Luciano *et al.*, "Digitized spiral drawing: a possible biomarker for early Parkinson's disease," *PLoS ONE*, vol. 11, no. 10, e0162799, 2016.
- [11] N. H. de Jong and T. Wempe, "Praat script to detect syllable nuclei and measure speech rate automatically," *Behav. Res. Methods*, vol. 41, no. 2, pp. 385–390, 2009.
- [12] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, "Detecting outliers: do not use standard deviation around the mean, use absolute deviation around the median," *J. Exp. Soc. Psychol.*, vol. 49, no. 4, pp. 764–766, 2013.
- [13] S. W. Roberts, "Control chart tests based on geometric moving averages," *Technometrics*, vol. 1, no. 3, pp. 239–250, 1959.
- [14] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765–4774.
- [15] Altoida Inc., "Altoida – precision neurology platform." [Online]. Available: <https://altoida.com/> (accessed Sep. 2026).

- [16] Linus Health, "About our assessments." [Online]. Available: <https://linushealth.com/about-assessments> (accessed Sep. 2026).
- [17] Google, "Flutter documentation." [Online]. Available: <https://docs.flutter.dev/> (accessed Sep. 2026).
- [18] Google, "Firebase documentation: Authentication and Cloud Firestore Security Rules." [Online]. Available: <https://firebase.google.com/docs> (accessed Sep. 2026).
- [19] S. Binder, "Drift – reactive persistence library for Dart and Flutter." [Online]. Available: <https://drift.simonbinder.eu/> ; Zetetic, "SQLCipher." [Online]. Available: <https://www.zetetic.net/sqlcipher/> (accessed Sep. 2026).
- [20] Ministry of Electronics and Information Technology, Government of India, *The Digital Personal Data Protection Act, 2023* (Act No. 22 of 2023).



APPENDIX

A.1 Full source code: [GitHub/Colab link]

A.2 Weekly PBL progress log (Table A.1) and mentor sign-offs.

Table A.1: Weekly PBL Progress Log

Week	Activity	Output	Mentor sign-off
1–2	Problem study, literature survey, choice of target conditions	Review 0 presentation	
3	Requirements; selection of tasks and features	FR/NFR list (Tables 4.1, 4.2)	
4	Wireframes and navigation design	Low-fidelity screens	
5	Architecture, data model, Firebase project set-up	Design diagrams (Ch. 5)	
6	Authentication, consent and check-in screens	Login and onboarding flow	
7–8	Task modules and on-device feature extraction	Four task runners; Review 1	
9	Screening engine prototyped in Python with unit tests	12 passing unit tests	
10	Simulated cohort evaluation; thresholds calibrated	Tables 9.3, 9.5	
11	Dashboard, report, PDF export, sync	High-fidelity screens	
12	Testing and documentation	Review 2; this report	

A.3 Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
Madhav M S	50	50	Equal contribution
M Kiruthick Kannaa	50	50	Equal contribution

A.4 Project Structure and Source Listings

Figure A.1: Project Structure

```
lib/
├── main.dart                app entry, Firebase init, ProviderScope
├── app/ router.dart theme.dart l10n/
├── features/
│   ├── auth/ consent/ checkin/ dashboard/ trends/ report/
│   └── session/ tasks/ memory_task.dart reaction_task.dart
│       speech_task.dart spiral_task.dart
├── engine/ features.dart extractors/ baseline.dart screening_engine.dart
├── data/ local_db.dart (Drift) repository.dart sync_service.dart
├── services/ notifications.dart pdf_report.dart
└── test/ integration_test/ engine_lab/ (Python reference + simulation)
```

Reaction trial timing (Dart). Random foreperiod; taps before the stimulus are counted as anticipations.

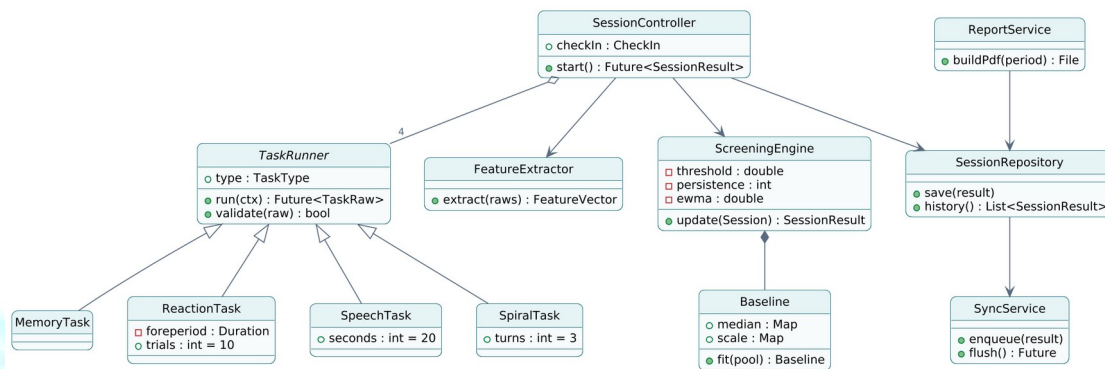
```
/// One trial: random 1–4 s foreperiod, then time from stimulus to tap.
Future<int?> runTrial() async {
  _tap = Completer<int>(); _armed = false; _early = false;
  await Future.delayed(Duration(milliseconds: 1000 + _rng.nextInt(3000)));
  if (_early) { anticipations++; return null; } // tapped before stimulus
  _armed = true; showStimulus(); _sw.reset().start();
  return _tap.future; // completed in onTap()
}

void onTap() {
  if (!_armed) { _early = true; return; }
  _sw.stop(); _armed = false; _tap.complete(_sw.elapsedMilliseconds);
}
```

Screening engine update (Dart port of the tested Python reference).

```
SessionResult update(Session s) {
  if (!s.valid) return SessionResult.invalid();
  if (baseline == null) return _addToBaselinePool(s); // skips first 2 + confounded
  if (s.confounded) return SessionResult.excluded();
  final z = {for (final d in Domain.values) d: _meanZ(s, d)}; // robust z, oriented
  final index = Domain.values
    .fold(0.0, (acc, d) => acc + weights[d]! * max(0.0, z[d]!));
  ewma = lambda * index + (1 - lambda) * ewma; // λ = 0.3
  run = ewma >= threshold ? run + 1 : 0;
  final status = run >= persistence ? Status.notable
    : ewma >= 0.6 * threshold ? Status.mild : Status.stable;
  return SessionResult(status, index, ewma, z, contributions(z));
}
```

Figure A.2: Class Diagram



A.5 Screening Engine Reference Implementation (Python)

The reference engine used for the unit tests and evaluation; the Dart engine in lib/engine/ mirrors it line for line.

```
@dataclass
class ScreeningEngine:
    threshold: float = 1.0 # EWMA level that counts as "notable"
    persistence: int = 3 # consecutive valid sessions above threshold
    use_context: bool = True # exclude self-reported confounded sessions
    use_ewma: bool = True
    history: list = field(default_factory=list)
    baseline: Baseline | None = None
    ewma: float = 0.0
    run: int = 0
    _seen: int = 0
    _pool: list = field(default_factory=list)
    def update(self, s: Session) -> dict:
        self._seen += 1
        if not s.valid:
            return {"status": "INVALID_SESSION"}
        if self.baseline is None:
            usable = self._seen > FAMILIARISATION_SESSIONS and not (self.use_context and
s.confounded)
            if usable:
                self._pool.append(s)
                if len(self._pool) >= BASELINE_SESSIONS:
                    self.baseline = Baseline.fit(self._pool)
                return {"status": "BUILDING_BASELINE", "progress": len(self._pool) /
BASELINE_SESSIONS}
            if self.use_context and s.confounded:
                return {"status": "EXCLUDED_CONTEXT"}
            z = domain_scores(s.features, self.baseline.median, self.baseline.scale)
            d = deviation_index(z)
            self.ewma = EWMA_LAMBDA * d + (1 - EWMA_LAMBDA) * self.ewma if self.use_ewma else d
            self.run = self.run + 1 if self.ewma >= self.threshold else 0
```

```
if self.run >= self.persistence:
    status = "NOTABLE_DEVIATION"
elif self.ewma >= 0.6 * self.threshold:
    status = "MILD_DEVIATION"
else:
    status = "STABLE"
return {"status": status, "index": d, "ewma": self.ewma,
        "domains": z, "contributions": contributions(z)}
```

A.6 Simulation Parameters

Table A.2: Simulated Population Parameters (mean, between-person SD, within-person SD)

Feature	Mean	Between SD	Within SD
Delayed recall (fraction)	0.75	0.12	0.06
Reaction median (ms)	320	45	18
Reaction CV	0.15	0.04	0.02
Speaking rate (wpm-equivalent)	140	20	8
Pause ratio	0.22	0.06	0.025
Spiral RMSE (dp)	6.0	1.5	0.6
Tremor index	0.10	0.03	0.012
Inter-key interval (ms)	260	50	15
Inter-key CV	0.35	0.08	0.035

Other settings: 60 sessions per user; practice effect 1.0 within-SD decaying with $\tau = 1.5$ sessions; tired days 15 % (effect 1.5 within-SD, reported 80 %); 3 % invalid sessions; decline onset at session 25, linear to 3 within-SD at session 60; calibration seed 11 (300 users/group), test seed 22 (500 users/group).